



ATLAS

Sistema de Suporte Técnico Remoto Multi-Operador (Co-browsing)

Documentação Técnica e Funcional Completa

Versão do documento: 1.0 · Junho/Julho de 2026

Criado por: Victor Silva & Pedro Cabral

Produção: <https://remote-assist-21.emergent.host>

Índice

1. Visão geral do produto
2. Personas e casos de uso
3. Arquitetura técnica
4. Fluxo de funcionamento (passo a passo)
5. Partilha de ecrã e co-browsing (WebRTC)
6. Miniaturas ao vivo no painel do técnico
7. Aplicação Android instalável (PWA)
8. Extensão Chrome (controlo avançado)
9. Autenticação e segurança
10. Modelo de dados (MongoDB)
11. API REST
12. Eventos Socket.io
13. Servidores STUN / TURN
14. Estrutura de ficheiros
15. Ambientes e deployment
16. Limitações conhecidas
17. Prompt completo de recriação do sistema

1. Visão geral do produto

Atlas é um sistema web de **suporte técnico remoto** pensado para clientes leigos. O cliente abre um link, toca em **COMEÇAR** e partilha o seu ecrã em tempo real. O técnico vê o ecrã do cliente através de um painel (CRM) e orienta-o com um **círculo vermelho** que replica os cliques do técnico no ecrã do cliente (co-browsing).

Cada técnico/operador tem o seu próprio link e o seu próprio CRM isolado. O sistema é totalmente transparente: o cliente vê sempre os avisos de autorização do sistema operativo e pode parar a partilha a qualquer momento.

Princípios de conceção

- **Simplicidade** Para o cliente: um único botão grande. Sem instalação obrigatória, sem registo.
- **Isolamento** Multi-operador: cada operador só vê as suas sessões (identificadas por `linkId`).
- **Tempo real** Sinalização e co-browsing via Socket.io; vídeo via WebRTC.
- **Transparência** O cliente controla o que partilha e vê sempre o pedido de autorização.

2. Personas e casos de uso

Persona	Dispositivo	Objetivo
Cliente leigo	Telemóvel/PC (mobile-first)	Receber ajuda: partilhar o ecrã e seguir o círculo vermelho do técnico.
Técnico / Operador	Desktop	Ver o ecrã do cliente, orientar com cliques, gerir sessões no CRM.
Administrador	Desktop	Conta com o mesmo painel; gestão das próprias sessões.

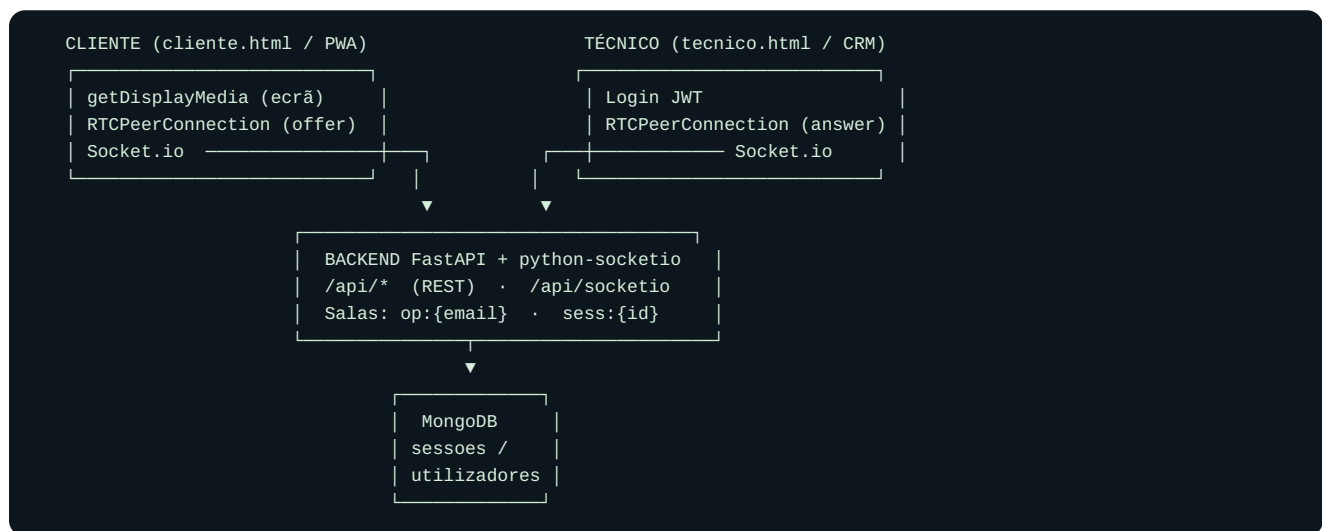
Casos de uso típicos

- Ajudar um cliente a configurar uma app, preencher um formulário ou resolver um erro no ecrã.
- Orientar passo a passo indicando onde tocar (círculo vermelho).
- Manter um histórico de acessos por operador (renomear/apagar sessões).

3. Arquitetura técnica

Camada	Tecnologia	Detalhe
Backend	FastAPI + python-socketio (ASGI)	Porta 8001; rotas REST sob /api ; Socket.io em /api/socketio .
Base de dados	MongoDB	Coleções sessoes e utilizadores .
Autenticação	JWT + bcrypt	Apenas para técnicos; contas semeadas a partir do .env .
Frontend	HTML/CSS/JS puro	Servido de frontend/public/ (mesma origem).
Vídeo	WebRTC	Cliente = offerer, técnico = answerer; STUN Google + TURN Metered.
App móvel	PWA	Manifest + service worker; instalável no Android.
Controlo avançado	Extensão Chrome (MV3)	Círculo e cliques reais em qualquer separador do navegador.

Diagrama lógico



4. Fluxo de funcionamento (passo a passo)

1. O técnico entra em `/tecnico.html`, faz login (JWT) e obtém o seu `linkId`.
2. O técnico partilha o link `/cliente.html?op=<linkId>` com o cliente.
3. O cliente abre o link. O socket liga-se e emite `cliente:hello` (op + token + userAgent).
4. O backend cria/recupera a sessão, responde `cliente:sessao` (id + token persistido em localStorage) e atualiza o CRM do operador (`sessoes:atualizar`).
5. O cliente toca em **COMEÇAR** → `getDisplayMedia` (pede autorização do SO) → emite `cliente:partilhar {ativo:true}`.
6. O técnico vê o cliente "Online / A partilhar". A ligação WebRTC é estabelecida: cliente cria a *offer*, o técnico responde com a *answer*; trocam candidatos ICE.
7. O técnico clica no vídeo → emite `tecnico:clique {x%, y%}` → o cliente desenha o círculo vermelho e simula o toque no ponto correspondente.
8. O cliente pode parar a partilha a qualquer momento; o técnico pode pedir reconexão.

5. Partilha de ecrã e co-browsing (WebRTC)

Sinalização

A negociação WebRTC é reencaminhada pelo Socket.io usando a sala `sess:{id}`. O cliente é sempre o *offerer* e o técnico o *answerer*. Os candidatos ICE são colocados numa fila (queue) até a descrição remota estar pronta, evitando o problema do “ecrã preto”.

Mapeamento de cliques (aware de letterbox)

Como o vídeo usa `object-fit: contain`, existem barras pretas. O técnico converte a posição do clique em percentagem **relativa ao conteúdo do vídeo** (ignorando as barras). O cliente reconverte a percentagem em píxeis do seu ecrã e dispara eventos `pointer/mouse/click` via `elementFromPoint`.

Robustez

- Fila de ICE + `video.play()` forçado para evitar ecrã preto.
- Screen Wake Lock API** para impedir que o ecrã do cliente adormeça durante a partilha.
- Deteção de partilha interrompida → ecrã “Reconectar?” com vibração.
- Correção de corrida `cliente:hello` vs `cliente:partilhar` (retry no servidor + gating no cliente).

6. Miniaturas ao vivo no painel do técnico

Assim que o painel abre, o técnico liga-se **automaticamente** a todos os clientes online que estão a partilhar e mostra a tela ao vivo dentro de cada cartão de sessão — *antes* de clicar em “Ver/Reconectar”.

- Arquitetura **multi-peer**: um mapa `pcs[sessaoId]` com uma `RTCPeerConnection` por sessão.
- O backend inclui o `sessaoId` nas mensagens `webrtc:offer` e `webrtc:ice` para o técnico encaminhar cada fluxo para o vídeo correto.
- Clicar na miniatura ou em “Ver/Reconectar” abre o modal **reutilizando o mesmo stream** (sem nova negociação).
- Fechar o modal mantém a miniatura ativa enquanto o cliente continuar a partilhar.

7. Aplicação Android instalável (PWA)

A página do cliente é uma **PWA** instalável no Android, com o visual Atlas, ícone próprio e ecrã de arranque (splash).

- `manifest.webmanifest`: nome “Suporte Atlas”, `display: standalone`, ícones 192/512 + *maskable*, cores de fundo/tema.
- `sw.js`: service worker mínimo (network-first na navegação; **nunca** interceta `/api` nem o Socket.io).
- Botão “**Instalar aplicação**” que surge via `beforeinstallprompt`.
- O `op` do operador é guardado em `localStorage` (`atlas_last_op`): a app instalada arranca em `/cliente.html` e recupera o operador.

Partilha no Android: no Chrome Android, ao tocar em COMEÇAR o telemóvel pede para partilhar o **ecrã inteiro** — o técnico consegue ver todas as apps.

Limitação da web no Android: o círculo vermelho/cliques só funcionam dentro da própria app de suporte (a web não pode desenhar sobre outras apps).

8. Extensão Chrome (controle avançado)

Extensão **Manifest V3** que permite mostrar o círculo do técnico e simular cliques **reais** em qualquer separador do navegador do cliente (não apenas na página de suporte).

Componente	Função
<code>offscreen.html/js</code>	Mantém a ligação Socket.io viva (o service worker MV3 adormece).
<code>background.js</code>	Recebe os cliques e executa cliques reais via <code>chrome.debugger</code> (<code>Input.dispatchEvent</code>).
<code>content.js</code>	Injetado em todos os sites: lê a sessão da página do cliente e desenha o círculo; mapeia % do ecrã → viewport da aba.
<code>popup.html/js</code>	Mostra o estado da ligação ao técnico.

Requisito: o cliente deve partilhar o **ecrã inteiro** para o mapeamento de coordenadas funcionar. A extensão não consegue clicar em apps nativas do SO (só dentro do navegador).

9. Autenticação e segurança

- **JWT** emitido no login; validado em cada pedido protegido (`Authorization: Bearer`).
- Palavras-passe com **bcrypt**. As contas são semeadas a partir do `.env` no arranque, com reposição de password caso divergente.
- Isolamento por operador: cada consulta/ação válida que a sessão pertence ao operador autenticado.
- O cliente **não** tem login; a sessão é identificada por um `token` aleatório persistido no dispositivo.

Boas práticas futuras (P2): rate-limiting / anti-força-bruta, rotação do `JWT_SECRET` , CORS restrito, consentimento explícito RGPD.

10. Modelo de dados (MongoDB)

Coleção **sessoes**

Campo	Tipo	Descrição
id	string (uuid)	Identificador da sessão.
token	string (uuid)	Token do cliente (persistido no dispositivo).
operador	string (email)	Operador dono da sessão.
nome	string	Nome amigável (só o técnico vê).
userAgent	string	Dispositivo/navegador do cliente.
inicio / ultimaAtividade	ISO datetime	Timestamps.
online / aPartilhar	bool	Estado em tempo real.
modo	string	"ecra".
sid	string	Socket atual do cliente.

Coleção **utilizadores**

Campo	Tipo	Descrição
email	string	Único; identificador de login.
senha_hash	string (bcrypt)	Hash da palavra-passe.
linkId	string	Identificador do link público do operador.
criado	ISO datetime	Data de criação.

11. API REST

Método	Rota	Descrição
GET	/api/	Health/identificação da app.
GET	/api/ice	Devolve os servidores STUN/TURN para o WebRTC.
POST	/api/auth/login	Login; devolve token JWT, email e linkId.
GET	/api/auth/me	Valida o token e devolve o utilizador.
POST	/api/auth/senha	Altera a própria palavra-passe.
GET	/api/sessoes	Lista as sessões do operador.
PATCH	/api/sessoes/{id}	Renomeia uma sessão.
DELETE	/api/sessoes/{id}	Apaga uma sessão.
DELETE	/api/sessoes	Apaga todas as sessões do operador.

12. Eventos Socket.io (/api/socketio)

Evento	Origem → Destino	Descrição
cliente:hello	Cliente → Servidor	Regista/recupera a sessão (op, token, userAgent).
cliente:sessao	Servidor → Cliente	Devolve id + token da sessão.
cliente:partilhar	Cliente → Servidor	Atualiza o estado aPartilhar (true/false).
sessoes:atualizar	Servidor → Técnico	Envia a lista de sessões do operador (sala op:{email}).
tecnico:ver	Técnico → Servidor	Entra na sala sess:{id} e avisa o cliente (tecnico:pronto).
tecnico:pronto	Servidor → Cliente	O técnico está pronto; o cliente cria a offer.
tecnico:pedir-reconexao	Técnico → Cliente	Pede ao cliente para retomar a partilha.
webrtc:offer / answer / ice	Cliente → Técnico	Sinalização WebRTC (com sessaoid).
tecnico:clique	Técnico → Cliente/Extensão	Coordenadas (x%, y%) do círculo/toque.
extensao:hello / ok / erro	Extensão → Servidor	Liga a extensão à sessão pelo token.

13. Servidores STUN / TURN

O endpoint `/api/ice` lê os servidores do `.env` e devolve-os ao frontend. O STUN (Google) chega para redes simples; o **TURN (Metered)** é essencial para ligações entre redes/NAT restritivos (evita o ecrã preto em produção).

Variável (.env)	Exemplo
STUN_URLS	stun:stun.l.google.com:19302, stun:stun.relay.metered.ca:80
TURN_URLS	turn:global.relay.metered.ca:80, ...,443, turns:...,443?transport=tcp
TURN_USERNAME / TURN_CREDENTIAL	credenciais Metered

Alterações às credenciais TURN só chegam à produção após **redesploy**.

14. Estrutura de ficheiros

```
/app
├── backend/
│   ├── server.py           # FastAPI + Socket.io (REST /api, sinalização, seeding)
│   ├── requirements.txt
│   └── .env                # MONGO_URL, JWT_SECRET, contas, STUN/TURN
├── frontend/public/
│   ├── index.html         # redireciona para /tecnico.html
│   ├── cliente.html/.js   # cliente (WebRTC offerer + PWA)
│   ├── tecnico.html/.js   # CRM (WebRTC answerer + miniaturas)
│   ├── estilos.css
│   ├── manifest.webmanifest # PWA
│   ├── sw.js              # service worker
│   ├── icons/             # ícones da PWA
│   └── privacidade.html
└── extension/             # Extensão Chrome MV3
    ├── manifest.json, background.js, content.js
    ├── offscreen.html/js, popup.html/js
    └── icons/, lib/socket.io.min.js
```

15. Ambientes e deployment

- **Preview (dev):** ambiente de desenvolvimento e testes.
- **Produção:** <https://remote-assist-21.emergent.host>

Alterações a `.env` (ex.: TURN) ou novas contas só chegam à produção após um **redploy**. A extensão Chrome exige reinstalação/republicação para atualizar.

16. Limitações conhecidas

- **iOS/Safari:** não suporta partilha de ecrã por web (comunicado ao cliente).
- **Apps de bancos:** muitas apps protegem o ecrã e aparecem a preto na captura.
- **Círculo/cliques na web:** só funcionam dentro da própria página/PWA; para outros separadores é necessária a extensão Chrome; nunca sobre apps nativas do SO.
- **TURN:** imprescindível entre redes diferentes; requer credenciais válidas em produção.

17. Prompt completo de recriação do sistema

Texto pronto a usar (por exemplo com um agente de IA) para reconstruir o Atlas de raiz:

Cria um sistema web de suporte técnico remoto multi-operador chamado "Atlas" para clientes leigos, com co-browsing.

OBJETIVO

O cliente abre um link, toca em "COMEÇAR" e partilha o ecrã em tempo real (WebRTC). O técnico vê o ecrã num painel (CRM) e orienta o cliente com um círculo vermelho que replica os cliques do técnico no ecrã do cliente. Cada operador tem link e CRM isolados. Transparência total: o cliente vê os avisos de autorização e pode parar quando quiser.

STACK

- Backend: FastAPI + python-socketio (ASGI combinado). Porta 8001. Rotas REST sob /api. Socket.io em /api/socketio.
- Base de dados: MongoDB (coleções "sessoes" e "utilizadores"), acedida via MONGO_URL e DB_NAME do .env.
- Auth: JWT + bcrypt (apenas técnico). Contas semeadas a partir do .env no arranque, com reposição de password se divergir. O cliente não tem login (token aleatório persistido em localStorage).
- Frontend: HTML/CSS/JS puro servido de frontend/public/ (mesma origem). O frontend usa sempre location.origin para chamar a API.
- WebRTC: cliente = offerer, técnico = answerer. STUN do Google + TURN (Metered), ambos lidos do .env e servidos por GET /api/ice. Fila de candidatos ICE até a descrição remota estar pronta (evita ecrã preto). Screen Wake Lock no cliente.

PÁGINAS

- /tecnico.html: login (JWT), CRM com sessões "Online agora" e "Histórico", link próprio do operador com botão copiar, renomear/apagar/limpar, alterar password. Cada cartão de sessão online mostra uma MINIATURA de vídeo ao vivo (multi-peer: um mapa pcs[sessaoId] de RTCPeerConnection). Clicar na miniatura ou em "Ver/Reconectar" abre um modal que reutiliza o mesmo stream; clicar no vídeo emite as coordenadas do clique em percentagem (aware de letterbox por object-fit: contain).
- /cliente.html?op=: mobile-first, um botão grande "COMEÇAR" (getDisplayMedia, frameRate ~10). Estados: inicio/ativo/erro/reconectar. Ao receber as coordenadas do técnico, desenha um círculo vermelho e simula pointer/mouse/click via elementFromPoint. É uma PWA instalável (manifest + service worker + ícones); guarda o "op" em localStorage para a app instalada arrancar sem o parâmetro no URL.

SOCKET.IO (eventos)

cliente:hello, cliente:sessao, cliente:partilhar, sessoes:atualizar, tecnico:ver, tecnico:pronto, tecnico:pedir-reconexao, webrtc:offer/answer/ice (com sessaoId), tecnico:clique, extensao:hello/ok/erro. Salas: op:{email} e sess:{id}. Cuidado com a corrida cliente:hello vs cliente:partilhar (o cliente só deve anunciar a partilha depois de receber cliente:sessao; o servidor deve tolerar/retry).

API REST

GET /api/, GET /api/ice, POST /api/auth/login, GET /api/auth/me, POST /api/auth/senha, GET /api/sessoes, PATCH /api/sessoes/{id}, DELETE /api/sessoes/{id}, DELETE /api/sessoes.

EXTENSÃO CHROME (MV3, opcional)

Permite círculo e cliques REAIS em qualquer separador do navegador do cliente: offscreen.html mantém o Socket.io; background.js usa chrome.debugger (Input.dispatchEvent); content.js injeta o círculo e mapeia % do ecrã para o viewport da aba (requer partilha do ECRÃ INTEIRO).

REQUISITOS DE QUALIDADE

- Design mobile-first, tom claro (verde Atlas), linguagem simples e honesta sobre limitações. Todos os elementos interativos com data-testid.
- Isolamento estrito por operador em todas as rotas/eventos.
- Nada de segredos hardcoded: tudo via .env. CORS configurável.